

动手学大模型应用开发 · 第 4 节

上下文工程 与记忆

让模型看到正确的信息

主讲：陈思州



理论主线 · 从本轮输入到长期信息治理

上下文工程不是把更多文字塞给模型，而是持续决定什么信息应在何时出现。

01 可见边界

上下文窗口、运行状态与外部记忆分别保存什么

02 推理与缓存

预填充、逐 Token 解码、KV Cache 与 Prompt Cache

03 选择和编排

Token 预算、稳定前缀、动态后缀与 GSSC 流水线

04 长程治理

记忆、压缩、按需读取、恢复与联合评估

先理解一次推理怎样读取信息，再讨论多轮任务怎样持续管理信息。

上下文是本轮采样实际包含的 Token

定义与边界 · 先检查模型实际收到什么

应用可访问的信息很多，但模型只能直接使用本轮请求中的内容。



没有进入请求的信息，不应被描述为模型已经记住或已经看到。

上下文、状态与外部记忆属于三个层次

信息分层 · 同一条事实可以位于不同存储层

判断问题时先问：信息现在存在哪里，谁能够读取，何时进入模型调用？

运行程序

维护状态，并在每次调用前组装上下文

01 模型上下文

本轮直接可见，调用时重新组装

02 任务状态

阶段、预算、观察和产物位置

03 外部记忆

跨轮次保存，使用前需要检索

04 原始资料

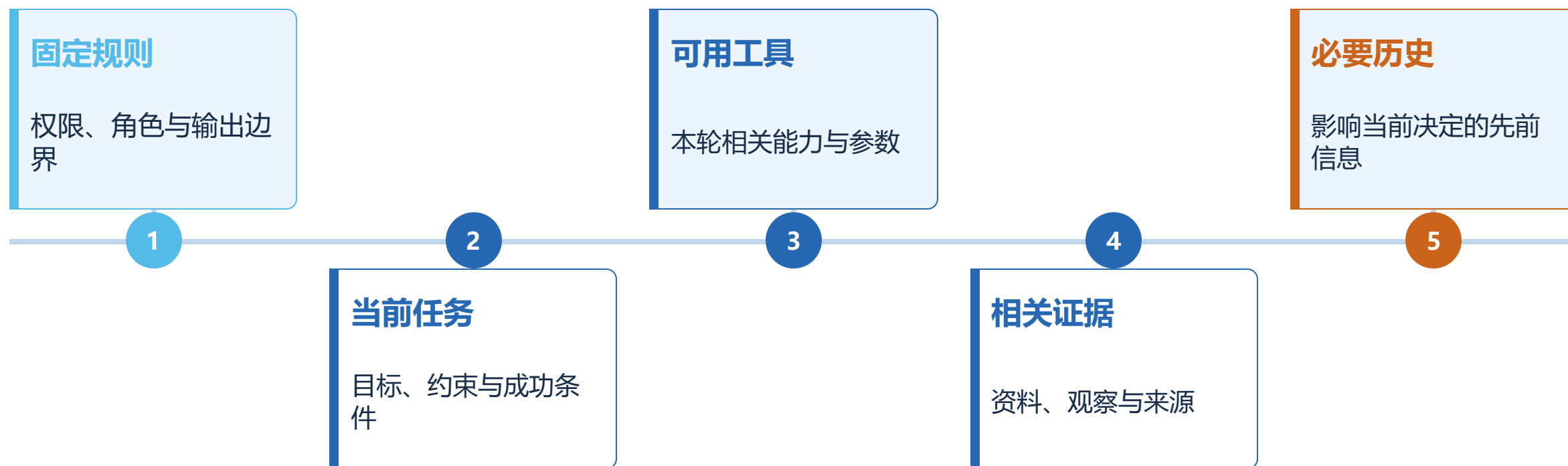
保留完整来源，按需读取或重新定位

状态不等于上下文，记忆也不是模型参数；模型只能使用被重新放入上下文的内容。

一次模型请求由五类信息组装而成

请求剖面 · 从稳定边界逐步加入任务相关信息

不同信息承担不同职责，不能用一段越来越长的聊天记录代替结构化组装。



上下文组装的标准不是“最近”，而是“是否影响当前决策并保持协议完整”。

一次生成包含预填充与逐 Token 解码

推理路径 · 输入先整体计算，输出再逐步生成

上下文长度主要影响预填充与缓存占用，输出长度继续增加解码步数。



减少输入和限制输出解决的是不同阶段的延迟与成本。

KV Cache 保存历史 Token 的注意力状态

推理状态 · 用内存换取逐 Token 解码速度

缓存避免重复计算旧 Token 的 K/V，但不会压缩语义，也不会修正无关上下文。



KV Cache 提高解码效率，但上下文选择仍决定缓存里装的是信号还是噪声。

A

单次生成的 KV Cache

服务当前请求的逐 Token 解码，缓存这次序列已经计算的 K/V 状态；请求结束后的生命周期由推理服务决定。

边界

B

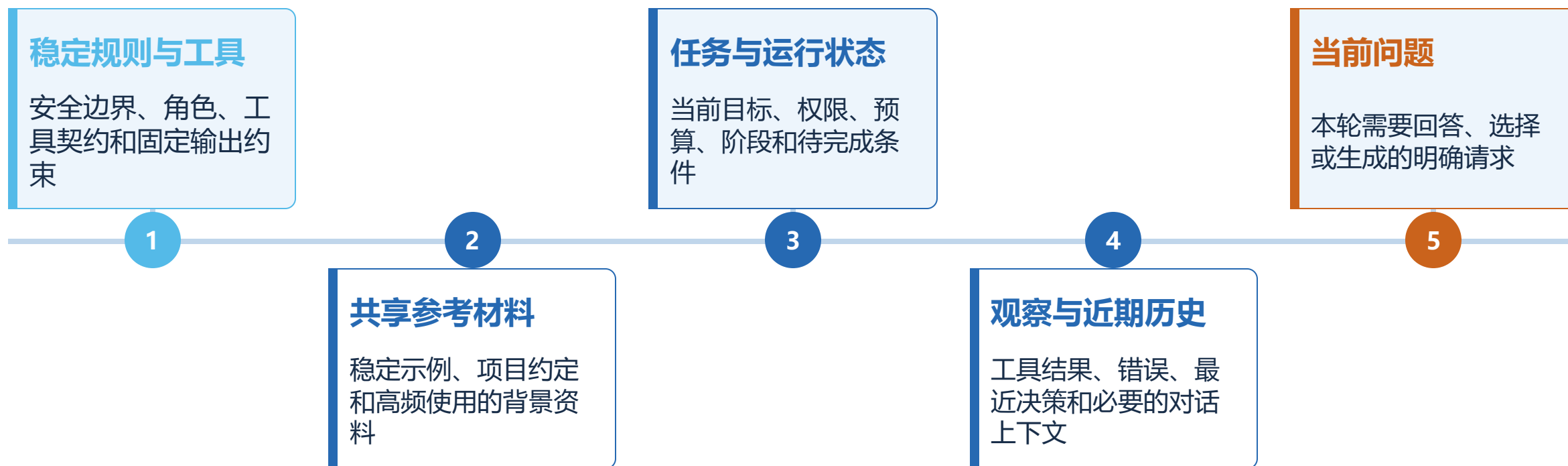
跨请求的 Prompt Cache

识别多个请求共享的稳定前缀，复用该前缀已经计算的状态；从第一个变化位置以后仍需重新处理。

智能体 Prompt 采用稳定前缀与追加后缀

Prompt 编排 • 先稳定、后动态，历史按发生顺序追加

逻辑顺序决定规则优先级、证据邻接关系和前缀缓存；实际字段由所用 API 定义。



把稳定内容留在前缀，把变化内容向后追加，兼顾可解释性与缓存复用。

上下文预算同时消耗容量、注意力、延迟与费用

有限资源 · 长度只是预算的一部分

同样的 Token 数量，信息密度、位置 and 任务相关性不同，实际价值也不同。

上下文预算

在成功概率与资源消耗之间做可解释取舍

01 窗口容量

输入、工具声明与输出共享上限

02 注意力

无关内容会竞争模型的处理资源

03 延迟

更长输入通常增加处理时间

04 费用

输入、输出和额外轮次共同计入

预算的目标是保留完成当前任务所需的最小充分信息，而不是机械压缩到最短。

同一段文字的字符数、UTF-8 字节数和 Token 数通常不同。

怎样理解

用 `len(text)` 统计 Python 字符长度。
用 `len(text.encode("utf-8"))` 统计字节。
模型 Token 用对应 tokenizer 或 usage 核对。

具体例子

"学习": 2 个字符, UTF-8 为 6 字节。
"AI": 2 个字符, UTF-8 为 2 字节。
JSON 还含字段名、标点和转义。
它的字符数不能直接充当 API Token 数。

本课装箱函数使用字符预算演示选择机制, 真实模型边界应使用对应 Token 计量。

Token 预算必须为工具回传与输出留出空间

预算账本 · 输入、后续观察与输出共享有限窗口

预算需要随任务阶段更新，不能在第一轮就把全部容量用完。

可用上下文预算

窗口上限减去输出预留、工具回传余量与安全边界

01 固定前缀

规则、工具契约、输出格式和稳定示例

02 任务证据

当前问题真正需要的资料、记忆与约束

03 运行轨迹

近期行动、工具结果、错误和状态变化

04 输出与工具余量

最终回答、结构化结果和下一轮观察的预留空间

好的预算会主动舍弃低价值输入，而不是等窗口溢出后随机截断。

窗口容量、信息质量与缓存命中不能混为一谈

四项指标 · 容得下、找得到、跑得动、复用得上

把指标分开，才能判断问题来自模型输入、注意力、服务内存还是请求编排。

上下文系统表现

质量、延迟、吞吐和成本由不同机制共同决定

01 窗口容量

一次请求允许的输入与输出 Token 上限

02 注意力质量

模型能否定位、组合并遵循关键内容

03 KV 内存

单请求序列与并发规模带来的推理内存压力

04 前缀命中

稳定内容能否跨请求复用已有计算状态

先确定哪项指标退化，再选择压缩、重排、缓存或模型侧优化。

工具请求与结果必须保持可关联的完整组

协议完整性 · 先保持行动与观察的对应关系

同一个工具可能多次调用，函数名不足以说明结果属于哪一次行动。

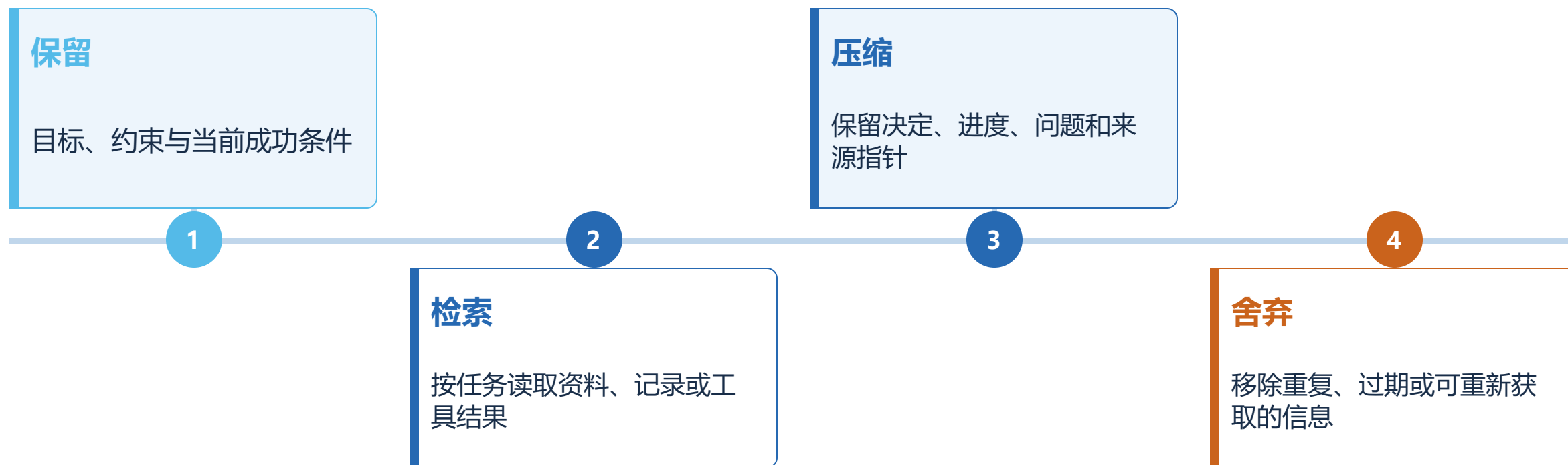
01	工具声明	定义能力、参数与返回语义
02	提出调用	模型生成名称、参数和调用标识
03	校验执行	程序检查权限、参数和预算
04	关联结果	结果绑定到对应调用标识
05	继续决策	下一轮把完整观察送回模型

协议合法是必要条件，但完整消息组仍可能缺少完成任务所需的语义前提。

上下文选择只有四类基本操作

选择策略 · 每条信息都应有明确的进入方式

有些信息需要常驻，有些旧内容更适合归档或按需取回。



好的选择策略同时记录选入内容、舍弃内容和重新取回信息的方法。

上下文构建可以拆成四个独立判断

构建流水线 • Gather、Select、Structure、Compress

四个阶段分别回答“有什么、要什么、怎么排、如何缩”，不把所有问题都归因于提示词。

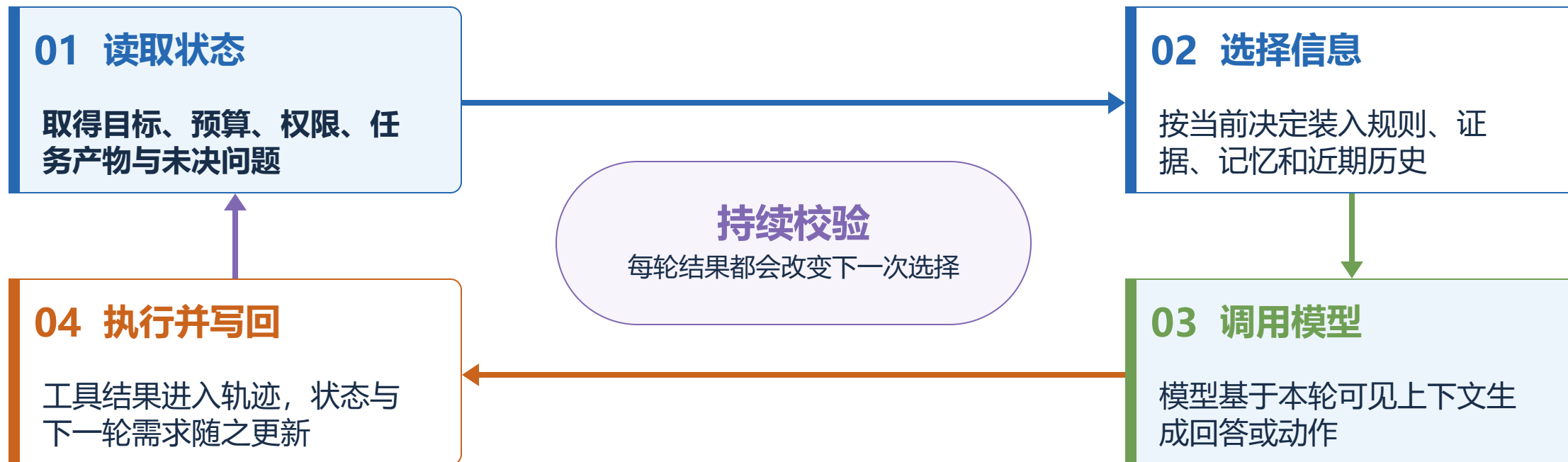
01	汇集 Gather	从系统规则、当前任务、运行状态、记忆和资料取得候选
02	筛选 Select	按当前决策、相关性、时效、权限和证据强度选择
03	组织 Structure	安排消息角色、工具关联、证据顺序与边界标记
04	压缩 Compress	减少重复内容，同时保留条件、未决问题和来源指针

把四步分别记录，才能定位漏取、误选、错排或压缩丢失。

智能体每一轮都会重新构建模型可见上下文

运行循环 · 上下文是每轮生成的视图

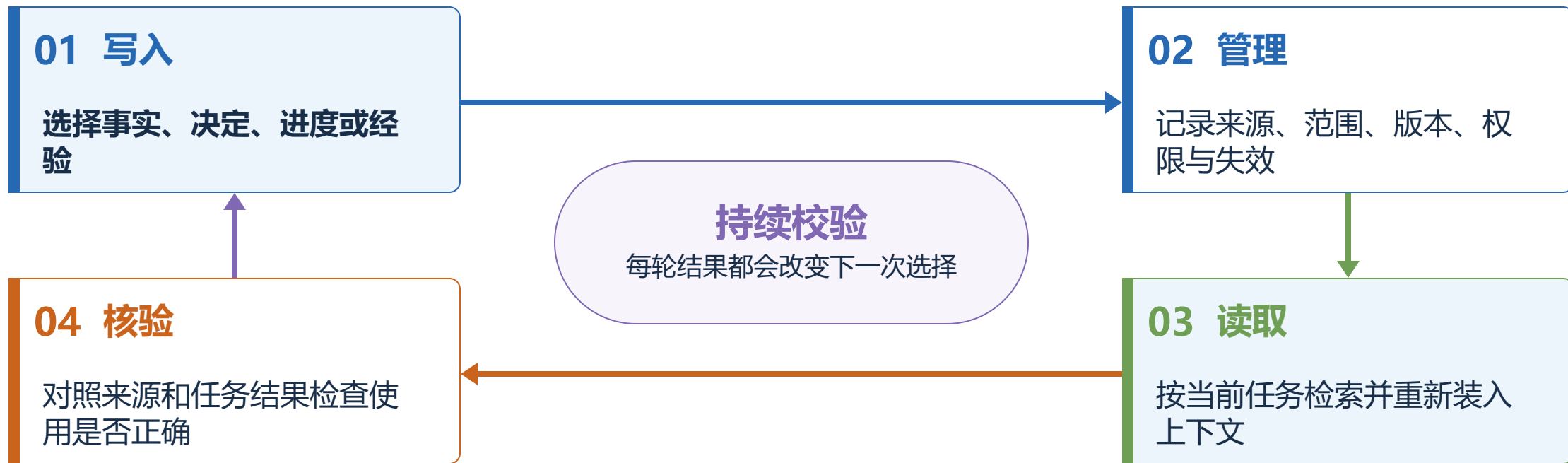
持久状态位于模型之外，模型只看到 Harness 为当前决定组装的那一部分。



多轮连续性由状态读取与写回维持，不靠模型永久记住全部历史。

记忆生命周期 · 保存成功不等于以后能正确使用

每一个环节都可能失败：写错、污染、召回错误、过期或被模型误读。



记忆的价值由后续任务效果证明，不由数据库中多了一条记录证明。

记忆写入需要来源、范围、版本与失效机制

写入治理 · 让记录可追溯、可隔离、可更新、可删除

存储字段不是越多越好，但每条长期记录都应说明自己的适用边界。

可用的记忆记录

内容与适用边界一起保存

01 来源

原始消息、文件、工具结果或人工确认

02 范围

用户、任务、项目与权限边界

03 版本

资料版本、更新时间与内容指纹

04 失效

过期、撤回、冲突与删除策略

长期记忆必须允许修订和遗忘，否则历史信息会逐渐变成当前任务的噪声。

摘要要保留决定下一步行为的条件，而不仅是主题词。

怎样理解

原要求：我想自己算，只给一个提示。
检查摘要是否保留学习目标与提示限制。
再制造字段齐全但题目记错的记录。

具体例子

摘要 A：学生在学习交叉熵。
主题正确，却丢了“只给提示”。
摘要 B：字段齐全，却要求给完整答案。
格式通过，含义仍不正确。

摘要检查至少包含完整性与忠实性。流畅的文字不能替代对原文的核查。

截断、压缩与外部化改变的信息不同

上下文缩减 · 每种方法损失的证据不同

缩短 Prompt 只是结果，关键在于还能否恢复条件、来源与未决问题。

缩减策略选择

比较信息损失、可恢复性、延迟与缓存影响

01 截断

按位置直接删除，适合明确过期且可丢弃的内容

02 摘要压缩

重写为短表示，需优先保护条件、异常和未决问题

03 工具结果清理

删除深历史原文，保留结论、状态和来源指针

04 外部化

把资料与笔记移出窗口，按需重新读取原始证据

不能恢复的压缩等于永久遗忘；可回查指针决定缩减是否安全。

比较输入策略时，固定模型、系统指令和当前问题。

怎样理解

完整历史：保留原讨论。

最新消息：只取结尾几条。

结构记忆：选取经核验的任务状态。

具体例子

统一问句：“继续给我提示”。

有原题时，检查提示是否针对原题。

缺原题时，检查是否先请求澄清。

用量和任务表现一起比较。更短的请求若丢了问题，不能视为优化成功。

长时程上下文 · 用外部记录保持连续性

笔记保存可恢复状态，不替代原始证据，也不自动成为事实。

结构化任务笔记

窗口之外的人类可读记录，支持版本控制与按需回注

01 当前状态

任务阶段、已经完成的工作和当前产物

02 核验结论

明确来源、版本和置信边界的已确认事实

03 阻塞与风险

缺失信息、失败原因、待确认假设和权限限制

04 下一步行动

可执行步骤、完成条件和需要重新读取的来源

新窗口先恢复状态，再按指针取回证据；不要把整段旧对话原样塞回。

设计流程 · 从“下一步需要判断什么”反推输入

上下文工程是一项循环工作，每次状态变化都可能改变下一轮需要的信息。

01	明确决策	模型下一步需要回答或选择什么
02	列出信息	目标、约束、证据、观察与工具
03	选择机制	保留、检索、压缩或舍弃
04	组装记录	检查协议、来源与预算并保存清单
05	结果验证	从任务效果判断信息是否足够

上下文问题应从真实请求和任务结果定位，而不是从模型是否“看起来聪明”判断。

评估框架 · 不能只看上下文长度或单次回答

优化一项指标可能伤害另一项：更短可能缺证据，更多记忆可能增加污染。

上下文与记忆设计

用任务结果与治理证据共同验收

01 任务效果

正确性、完整性、遵循约束与拒答

02 资源成本

Token、延迟、调用次数与存储

03 记忆质量

写入正确、召回相关、冲突与过期

04 安全治理

来源、权限、隔离、删除与审计

用同一任务集比较策略，并保存请求、结果、用量和记忆操作，才能解释差异。

知识结构 · 从本轮输入到长期信息治理

上下文工程决定这一轮看什么，记忆系统决定哪些信息以后还能被可靠取回。



可见边界

上下文、运行状态与
外部记忆分层



推理成本

预填充、KV
Cache、Prompt
Cache 与预算



信息循环

GSSC、写入、读
取、压缩与恢复



联合验收

效果、成本、缓存、
记忆质量与安全

上下文工程既决定模型看见什么，也决定这些信息怎样被计算、复用和持续维护。

本节关于上下文选择、推理缓存、长程记忆与评估主要参考以下论文和官方资料。

- | | | |
|-----|-------------------------|---|
| [1] | Anthropic | Effective Context Engineering for AI Agents
anthropic.com/engineering/effective-context-engineering-for-ai-agents |
| [2] | OpenAI | Prompt Caching
developers.openai.com/api/docs/guides/prompt-caching |
| [3] | Anthropic | Prompt Caching
platform.claude.com/docs/en/build-with-claude/prompt-caching |
| [4] | Kwon, Woosuk, et al. | Efficient Memory Management for Large Language Model Serving with PagedAttention
SOSP 2023 · arXiv:2309.06180 |
| [5] | Liu, Nelson F., et al. | Lost in the Middle: How Language Models Use Long Contexts
TACL 2023 · arXiv:2307.03172 |
| [6] | Packer, Charles, et al. | MemGPT: Towards LLMs as Operating Systems
arXiv:2310.08560 |
| [7] | OpenAI | Agents SDK: Context Management
openai.github.io/openai-agents-python/context |
| [8] | Du, Pengfei | Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers
arXiv:2603.07670 |